

MODELIZACIÓN DE PROBLEMAS DE LA EMPRESA:

Intro RAG Bake-Off for Technical Q&A

Shadi Fedriani Abdallah
Alejandro Domínguez López

¿QUÉ HEMOS APRENDIDO?

Al comenzar este proyecto, nuestra experiencia en programación se limitaba principalmente a lo aprendido durante la carrera de matemáticas, por lo que adentrarnos en el mundo de los LLMs y el RAG supuso una curva de aprendizaje inmensa que nos permitió ganar fluidez técnica.

Uno de los primeros descubrimientos prácticos fue enfrentarnos a la realidad de las limitaciones técnicas; al elegir el modelo, nos dimos cuenta de que la potencia bruta no era el único factor, sino que los límites de frecuencia (RPM) dictaban la viabilidad del sistema, llevándonos a elegir Gemini 2.5-flash-lite por sus 1000 peticiones diarias.

Modelo	RPM	TPM	RPD
Modelos de texto de salida			
Gemini 2.5 Pro	2	125,000	50
Gemini 2.5 Flash	10	250,000	250
Versión preliminar de Gemini 2.5 Flash	10	250,000	250
Gemini 2.5 Flash-Lite	15	250,000	1,000
Versión preliminar de Gemini 2.5 Flash-Lite	15	250,000	1,000
Gemini 2.0 Flash	15	1,000,000	200
Gemini 2.0 Flash-Lite	30	1,000,000	200

Al establecer el baseline, nos sorprendió ver que el modelo alcanzaba una precisión alta sin ningún documento de apoyo, lo cual nos enseñó que los LLMs son capaces de deducir respuestas correctas basándose en patrones de formulación y lógica general, incluso sin conocimiento específico del paper.

Sin embargo, el aprendizaje más crítico surgió en el preprocesamiento, donde comprobamos empíricamente que la calidad del sistema depende mayoritariamente de la limpieza de los datos ; descubrimos que elementos como las referencias bibliográficas actuaban como "trampas" semánticas al contener títulos similares al tema principal, lo que nos obligó a implementar una limpieza estricta eliminando líneas numéricas o demasiado cortas para evitar confundir al modelo con datos sin contexto.

También comprendimos que la estrategia de chunking no podía ser arbitraria, rechazando la división por caracteres fijos en favor de un chunking semántico con solapamiento para asegurar que las definiciones no quedaran cortadas a la mitad, impidiendo la recuperación correcta.

En cuanto a la recuperación, observamos una dicotomía fascinante: BM25 funcionaba extraordinariamente bien, debido a que el documento estaba repleto de acrónimos técnicos

únicos como "REFRAG" o "TTFT", actuando casi como un filtro perfecto de ruido. El Dense Retrieval, aunque teóricamente superior al entender significados, sufría de "deslizamiento semántico" en preguntas de datos duros, recuperando párrafos temáticamente relacionados pero que carecían del dato numérico exacto.

Esto nos llevó a valorar el Hybrid Retrieval, que, aunque introducía mayor coste computacional al procesar la consulta dos veces, aseguraba mediante el recall que el fragmento relevante estuviera disponible para el LLM ante la variabilidad del lenguaje humano.

Finalmente, entendimos que en un entorno real la eficiencia es clave, notando que el context overhead en los métodos densos e híbridos era significativamente mayor que en BM25, lo que nos enseñó que a veces una solución ligeramente menos precisa pero más rápida y barata puede ser la óptima.

PREPROCESAMIENTO DE DATOS

En la fase de preprocesamiento, hemos comprobado empíricamente el principio de "garbage in, garbage out". La calidad de vuestro sistema RAG dependía casi exclusivamente de la calidad de los fragmentos (chunks). Trabajar con un PDF científico como el de REFRAG presentó desafíos estructurales que un simple pdf_to_text no podía resolver. Detectamos que algunas secciones como las Referencias, introducían un ruido semántico considerable. Las referencias son especialmente peligrosas en RAG porque contienen títulos de otros artículos que son semánticamente muy similares al tema principal, actuando como "trampas" para el método.

Para solucionar esto, implementamos un pipeline de limpieza bastante estricto (algo rígido). Primero, eliminamos líneas que contenían exclusivamente datos numéricos o que tenían una longitud inferior a 20 caracteres. Esto fue crucial para filtrar los datos crudos de las tablas y gráficos del paper, que al perder su formato tabular en la conversión a texto, se convertían en secuencias de números sin contexto que solo confundían al modelo de embeddings. Aunque esto supuso el riesgo de perder algún subtítulo, la ganancia en la "densidad de información" de los párrafos restantes fue superior.

4.1 Ablation Study Curriculum learning is essential for effective training in the reconstruction task. The reconstruction task, while intuitive, is particularly challenging when multiple chunks must be reconstructed. Table 11 shows the 2Percentage calculated as 2Percentage calculated as LLaMA-No Context-Log-Perplexity to Inspect LLaMA-No Context-min(LLaMA-Full Context, LLaMA-32K) 5 Table 11 Log-Perplexity on output tokens $x_{s+1:s}$ given context tokens $x_{1:s}$ for different models. We use $s = 2048$ and $o \in \{512, 1024, 2048\}$ here. Bolding are based on comparing baselines excluding LLaMA-Full Context and LLaMA-32K since they are expected to be the best (ideally). The lower the better (↓). LLaMA-Full Context LLaMA-32K LLaMA-No Context LLaMA256 REFLUG CEPE REFRAG8 REFRAG16 REFRAG32 Arxiv P512 P1024 P2048 P512 P1024 P2048 ProofFile P1024 P2048 LLaMA-Full Context LLaMA-32K LLaMA-No Context LLaMA256 REFLUG CEPE REFRAG8 REFRAG16 REFRAG32 Context Length ~4096 Context Length ~1024 Arxiv Book P519 ProofFile Arxiv Book P519 ProofFile Arxiv Book P519 ProofFile LLaMA-Full Context LLaMA-32K LLaMA-No Context LLaMA256 REFLUG CEPE REFRAG8 REFRAG16 REFRAG32 Context Length ~4096 Context Length ~1024 Arxiv Book P519 ProofFile Arxiv Book P519 ProofFile Reconstruction task is essential for the model to learn the continual pre-training task. Table 12 shows the performance of the continual pre-training task with and without initialization from the reconstruction task. The results indicate that pre-training on the reconstruction task is important for the success of continual pre-training. Advantages of RL-based selective compression. Figure 3 under various compression rates, achieved by varying the number of chunks to compress (i.e., adjusting g). Notably, a compression rate of 8 can be obtained either by configuring REFRAG16 to compress the appropriate number of chunks, or by employing REFRAG8 with full compression, which is natively trained at a compression rate of 8. This raises a natural question: does the former approach outperform the latter? Table 13 demonstrates that REFRAG16 with RL-based selective compression consistently outperforms REFRAG8 across different datasets and context lengths. Figure 3 Log-Perplexity on $x_{s+1:s}$ under varying compression rates by selectively compressing different percentages of chunks. We compare three selection methods: RL (trained policy), Perplexity-desc (heuristic: lower perplexity), Perplexity-asc (heuristic: higher perplexity), and Random (random selection).	Antes
LLaMA-Full Context LLaMA-32K LLaMA-No Context LLaMA256 REFLUG CEPE REFRAG8 REFRAG16 REFRAG32 Arxiv P512 P1024 P2048 1.069 1.074 1.075 1.076 1.084 1.086 1.254 1.371 1.526 1.171	Después

Posteriormente, abordamos la estrategia de 'chunking'. Rechazamos la división simplista por caracteres fijos porque corta ideas a la mitad. En su lugar, adoptamos un chunking semántico. Esta técnica intenta dividir el texto primero por separadores lógicos fuertes (dobles saltos de línea para párrafos), luego por frases y finalmente por palabras, solo si es necesario. Definimos un tamaño de chunk objetivo de ~250 tokens con un solapamiento ('overlap') de 30 tokens. Decidimos implementar solapamiento porque si no, el retriever podría recuperar la mitad de la definición y el LLM sería incapaz de inferir la respuesta correcta. Insertamos ejemplo de un chunk donde se aprecia el solapamiento:

```
--- CHUNK 14 (182 palabras) ---
capturing this diversity within a fixed-length embedding presents a significant challenge. Additionally, reconstructing  $s = k \times L$  tokens from  $L$  chunk embeddings further compounds the difficulty of the task. Counterintuitively, directly continuing pre-training of the decoder to utilize encoder outputs did not reduce perplexity, even for the reconstruction task. To address the optimization challenge, we propose employing curriculum learning for both tasks. Curriculum learning incrementally increases task difficulty, enabling the model to gradually and effectively acquire complex skills. For the reconstruction task, training begins with reconstructing a single chunk: the encoder receives one chunk embedding  $c_1$  for  $x_{1:k}$  and the decoder reconstructs the  $k$  tokens using the projected chunk embedding  $ec_{c_1}$ . Subsequently, the model reconstructs  $s$ , and so forth. To continuously adjust task difficulty, we vary the data mixture over time, starting with examples dominated by easier tasks (e.g., single chunk embedding) and gradually shifting towards those dominated by more difficult tasks (i.e.,  $L$  chunk embeddings). A visualization of the data mixture during curriculum learning is provided in figure 6, and the detailed scheduling is presented in table 8.
```

Finalmente, indexamos estos chunks limpios en una lista primero, y luego en la base de datos vectorial (ChromaDB) y en el BM25 Corpus, transformando el texto no estructurado en una base de conocimiento accesible.

```
# BM25 corpus
tokenized_corpus = [better_tokenize(chunk) for chunk in chunks]
bm25 = BM25Okapi(tokenized_corpus)

#Embeddings

def build_vector_collection(chunks, name="refrag_collection"):
    ef = embedding_functions.SentenceTransformerEmbeddingFunction(
        model_name="all-mpnet-base-v2"
    )
    client = chromadb.Client()
    try:
        client.delete_collection(name=name)
    except:
        pass
    collection = client.create_collection(
        name=name,
        embedding_function=ef,
        metadata={"hnsw:space": "cosine"}
    )
    ids = [str(i) for i in range(len(chunks))]
    collection.add(documents=chunks, ids=ids)
    return client, collection

chroma_client, chroma_collection = build_vector_collection(chunks, name="refrag_collection")

reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')
```

PRECISIÓN DE LAS RESPUESTAS

El análisis de la precisión de las respuestas reveló la naturaleza de los LLMs. Comenzando con el LLM Baseline, observamos una tasa de acierto sorprendentemente alta, cercana al 73%. Dado que el examen tenía 4 opciones, la aleatoriedad dictaría un 25%. Este resultado no implica que el modelo conociera el paper de REFRAG (comprobamos que gemini 2.5-flash-lite está entrenado antes del lanzamiento del paper), sino que demuestra la poderosa capacidad de razonamiento deductivo del modelo. El LLM fue capaz de descartar opciones absurdas o identificar la respuesta más probable basándose en la estructura gramatical o en su conocimiento general sobre informática.

RESULTADO BASELINE

Aciertos: 51 de 70

Precisión Final: 72.85714285714285%

Fallos en preguntas: [11, 18, 21, 22, 25, 26, 33, 37, 38, 39, 40, 44, 47, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70]

La implementación de BM25 supuso un salto cualitativo enorme, alcanzando precisiones aproximadas al 93%. El número de fragmentos que enviábamos cambiaba radicalmente el porcentaje de acierto del modelo (lo cual nos inspiró a medir el recall@k, que explicamos un poco más adelante).

RESULTADO BM25

Aciertos: 65 de 70

Precisión Final: 92.85714285714286%

Fallos en preguntas: [17, 38, 44, 63, 69]

Por contra, el Dense Retrieval mostró un comportamiento mixto. Aunque teóricamente superior por entender significados, a veces fallaba en preguntas de precisión. Si la pregunta pedía un dato numérico específico, el modelo de embeddings podía recuperar párrafos genéricos porque semánticamente son cercanos, pero que no contenían el dato exacto. Las precisiones mejoraron levemente al BM25 con un 96%

RESULTADO DENSE RETRIEVAL

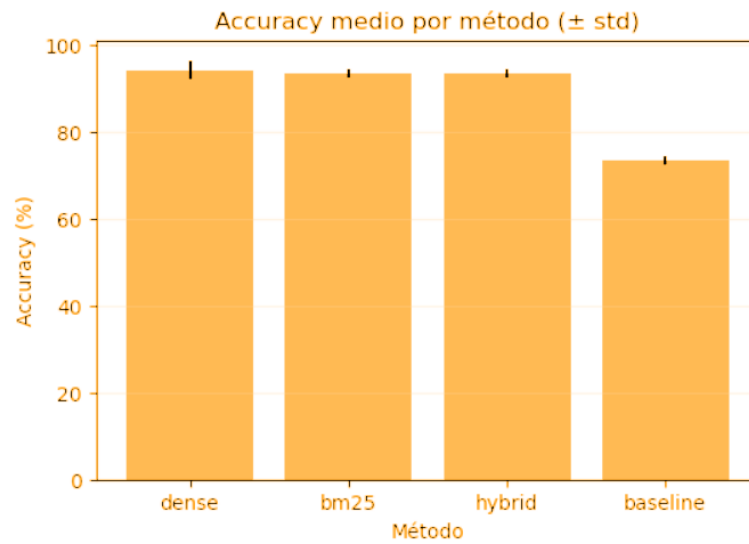
Aciertos: 67 de 70

Precisión Final: 95.71428571428572%

Preguntas falladas (índices): [3, 38, 44]

Luego, al implementar Hybrid Retrieval, no hemos sido capaces de optimizar su precisión, con la diferencia de que requería mucha más complejidad computacional, de hecho, suele dar como precisión una media aritmética entre ambas.

Comprobando dónde fallaba cada método vimos que había unas preguntas del dataset que no conseguían acertar ninguno de los tres: la 38 y la 44, que eran las que hacían que el modelo no tuviera más precisión.



	method	mean	std	min	max
2	dense	94.285714	2.020305	92.857143	95.714286
1	bm25	93.571429	1.010153	92.857143	94.285714
3	hybrid	93.571429	1.010153	92.857143	94.285714
0	baseline	73.571429	1.010153	72.857143	74.285714

MÉTRICAS DE RECUPERACIÓN

Para evaluar el funcionamiento de nuestro sistema, no nos bastó con saber si el LLM acertaba; necesitábamos saber si recuperaba la evidencia correcta. Aquí introducimos métricas de recuperación clásicas: Source Attribution Accuracy y Recall@K.

La atribución de fuente verifica si el fragmento citado por el modelo ("paper_reference") coincide realmente con el chunk que contiene la respuesta. Notar que le pusimos un 75% de umbral de coincidencia para que si al limpiar el texto se modificaba algo pequeño del párrafo como un espacio, punto, etc. no lo penalizara por completo.

Nuestros datos mostraron que BM25 dominaba el Source Attribution en preguntas con "wording" (redacción) similar al texto. Si la pregunta copiaba una frase del paper, BM25 encontraba el párrafo origen con una precisión casi del 100%. Sin embargo, su Recall (capacidad de encontrar el documento relevante) caía en picado cuando la pregunta usaba sinónimos. Esto nos enseñó que la búsqueda semántica introduce un "deslizamiento semántico": trae contenido relacionado temáticamente, pero no siempre el dato exacto necesario para una pregunta de tipo test. Aquí fue cuando nos dimos cuenta de que sin un mecanismo de re-ranking (reordenamiento) posterior, los embeddings pueden ser menos precisos que la búsqueda por palabra clave para tareas de extracción de datos duros.

PIPELINE	PRECISIÓN	COINCIDENCIA PROMEDIO
BM25	92.86%	97.1%
DENSE	85.71%	94.0%
HYBRID	90.00%	96.1%

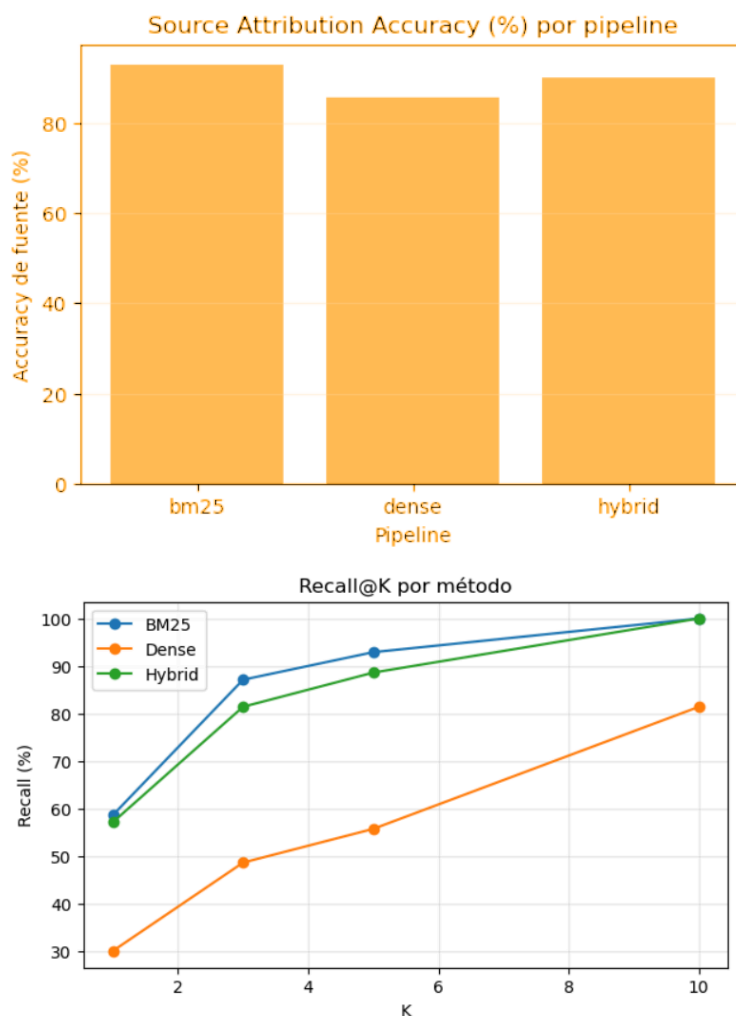
GANADOR: BM25 con un 92.86% de precisión de fuente.

En recall@k podemos ver que para k=1, k=3, k=5 el BM25 superó a los otros dos métodos, ¿pero por qué? La respuesta reside en la naturaleza de los modelos. El paper de REFRAG está lleno de términos técnicos únicos y acrónimos específicos: "REFRAG", "CEPE", "TTFT", "Meta Superintelligence Labs". BM25 valora enormemente la aparición de palabras raras. Cuando una pregunta contenía el término "REFRAG", BM25 filtraba inmediatamente cualquier chunk que no contuviera esa palabra exacta, actuando como un filtro de ruido muy eficaz.

Métrica	BM25	Dense	Hybrid	Ganador
Recall @ 1	58.6%	30.0%	57.1%	BM25
Recall @ 3	87.1%	48.6%	81.4%	BM25
Recall @ 5	92.9%	55.7%	88.6%	BM25
Recall @ 10	100.0%	81.4%	100.0%	Hybrid

El Hybrid Retrieval brilló en las métricas para k=10. Aunque a veces el chunk correcto no era el primero, el Hybrid aseguraba casi siempre que el chunk relevante estaba dentro de los 5 primeros resultados enviados al LLM. Esto valida la hipótesis de que la búsqueda vectorial es

más robusta ante la variabilidad lingüística humana. Al combinar ambos métodos, el Hybrid logró la mayor consistencia: usaba BM25 como "ancla" para términos técnicos y vectores densos para capturar el contexto, maximizando la probabilidad de que el LLM recibiera la información necesaria para responder, independientemente de cómo se formulara la pregunta. Esto es crítico en aplicaciones reales donde los usuarios no son expertos y usan lenguaje natural coloquial.



EFICIENCIA COMPUTACIONAL

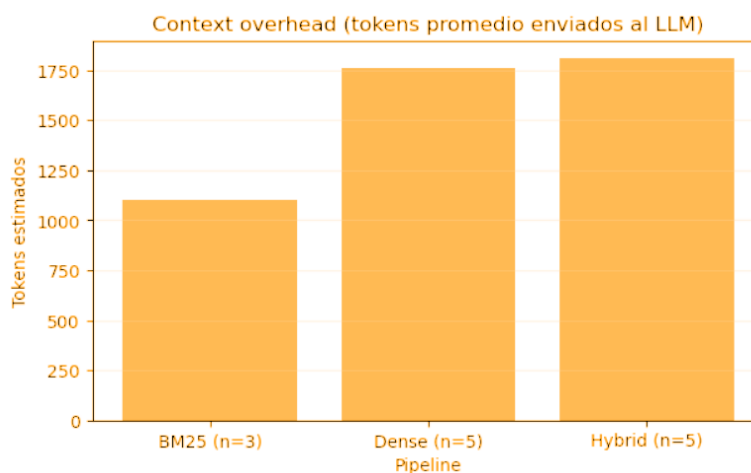
Una dimensión a menudo olvidada en los proyectos académicos, pero vital en el mundo empresarial, es el coste. Definimos el Context Overhead como la cantidad de tokens "inútiles" que enviamos al LLM para responder una pregunta. Cada token tiene un coste monetario y de latencia.

El pipeline de BM25 resultó ser el más eficiente en términos de overhead. Al ser tan selectivo con las palabras clave, podíamos permitirnos enviar solo el top-3 de chunks, sabiendo que eran muy probablemente relevantes. Esto mantenía el prompt corto y la respuesta rápida.

Por el contrario, las estrategias de Dense Retrieval y Hybrid mostraron un overhead mayor, de entre 1.6x y 2x respecto a BM25. La razón es técnica: para compensar la falta de precisión exacta de los embeddings, tuvimos que aumentar el número de chunks recuperados (Top-5 o Top-10) para asegurar el Recall. Además, el sistema Híbrido requiere procesar la consulta dos veces (una por BM25, otra por vectores) y luego fusionar los resultados, lo que añade latencia al sistema antes incluso de llamar al LLM.

Pipeline	Avg Caracteres	Est. Tokens	Coste Relativo
BM25 (n=3)	4416	1104	1.0x (Base)
Dense (n=5)	7040	1760	1.6x
Hybrid (n=5)	7230	1807	1.6x

Este análisis nos conecta directamente con el tema del paper: el problema de la latencia en contextos largos. Al igual que REFRAG propone comprimir el contexto para mejorar el TTFT (Time-To-First-Token), nosotros aprendimos que una mala estrategia de recuperación obliga a enviar contextos gigantescos al LLM para compensar, lo que ralentiza la generación y dispara los costes. En un entorno real, no siempre buscamos la máxima precisión (accuracy) a cualquier coste; a veces, un sistema BM25 que es un 2% menos preciso pero un 50% más barato y rápido es la solución óptima de negocio.



CONCLUSIONES

Tras analizar a fondo los resultados, nos hemos dado cuenta de que implementar un RAG no es una solución mágica que sirva igual para todo, sino más bien un abanico de decisiones de diseño donde cada detalle cuenta. Hemos aprendido que la limpieza de datos o preprocesamiento influye mucho más en el resultado final que el propio modelo de lenguaje; un fragmento mal cortado o una referencia bibliográfica que se cuele pueden arruinar la recuperación, por muy potente que sea el LLM.

Respecto a la estrategia de recuperación, hemos llegado a una matriz de decisión bastante clara: por un lado, BM25 es el rey en dominios de jerga estricta o vocabulario único, como vimos con el paper de Meta lleno de siglas raras como "REFRAG", "CEPE" o "TTFT" . Al basarse en la rareza de las palabras, si una pregunta tiene uno de estos términos, BM25 filtra el ruido casi a la perfección, actuando como un "Ctrl+F" inteligente ideal para entornos donde se buscan códigos exactos o referencias específicas y donde la latencia importa, ya que es rapidísimo y no pide GPU . Eso sí, falla estrepitosamente si usamos sinónimos; si el paper dice "latencia" y preguntamos por "retardo", no encontrará nada.

Ahí es donde entra el Dense Retrieval, que es perfecto para consultas más vagas o conceptuales, como preguntar por la velocidad sin usar la palabra técnica exacta, ya que entiende que vectorialmente los conceptos están cerca . Es ideal para usuarios no expertos o incluso para cruzar idiomas, aunque tiene el peligro de sufrir alucinaciones, trayendo cosas que suenan parecidas pero no lo son .

Finalmente, el Hybrid Retrieval nos pareció la opción para no perder nada (maximizar recall) y dar robustez ante cualquier tipo de usuario, ya sea experto o novato, especialmente si le añadimos una capa final de Re-ranking para quedarnos con lo mejor de ambos mundos .

Más allá de las métricas, este proyecto nos ha enseñado que el verdadero cuello de botella hoy en día no es el cerebro (el modelo), sino la biblioteca (el dato); un LLM potente puede ser muy convincente alucinando, por lo que el RAG es ese mecanismo de seguridad indispensable . Al final, la diferencia entre un asistente útil y uno que no sirve no estaba en usar el modelo más caro, sino en esas decisiones de ingeniería invisibles, confirmándonos que el futuro de la IA aplicada va más de construir mejores sistemas de recuperación para que los modelos consulten bien, que de simplemente entrenar cerebros cada vez más grandes .